

Apple Health Export

Projeto de Parsing, Performance e Engenharia em Python

Marcos Pereira / DevButter

2026

Sumário

1	Introducao: o que este projeto quer resolver	1
1.1	Por que este projeto e bom para estudo?	1
1.2	O que nao fazer	2
1.3	O que fazer	2
1.4	Meta de aprendizado	2
2	Mapa dos conceitos que o projeto vai treinar	3
2.1	I/O-bound	3
2.2	CPU-bound	3
2.3	GIL	4
2.4	Stream	4
2.5	Fila	4
2.6	Batching	4
2.7	Logging	5
2.8	Profiling	5
2.9	Big O aplicado	5
2.10	Resumo do mapa	5
3	Estrutura do projeto	7
3.1	Estrutura inicial fornecida	7
3.2	Estrutura agregada proposta	7
3.3	Por que cada pasta existe?	8
3.4	Regra de separacao	9
4	I/O e streaming: lendo arquivos grandes sem sofrer	10
4.1	O que e I/O?	10
4.2	Por que o XML gigante muda a estrategia?	10
4.3	Streaming com iterparse	10
4.4	Explicando cada nomezinho	11
4.5	Complexidade	11
4.6	Primeiro exercicio	11
5	Dataclass e modelagem de dados	13
5.1	Por que modelar antes de inserir no banco?	13
5.2	Modelo HealthRecord	13
5.3	Explicando cada nomezinho	13
5.4	Por que manter value e value_numeric?	14
5.5	Funcao safe_float	14
5.6	Exercicio	14

6	Postgres desde o inicio	16
6.1	Por que Postgres em vez de CSV?	16
6.2	Docker Compose para Postgres	16
6.3	Settings	17
6.4	Schema inicial	17
6.5	Tabela parse_runs	17
6.6	Indices	18
6.7	Por que indice importa?	18
7	Parser streaming + batching	19
7.1	Fluxo desta etapa	19
7.2	Parser de elemento Record	19
7.3	Repository com batch insert	20
7.4	Pipeline sincrona	21
7.5	Conceitos aplicados	22
7.6	Experimento	22
8	Filas e threads: producer/consumer	24
8.1	Por que usar fila?	24
8.2	Queue	24
8.3	Explicando cada nomezinho	24
8.4	Producer	24
8.5	Consumer	25
8.6	Rodando com Thread	25
8.7	Quando thread ajuda?	26
8.8	Experimento	26
9	GIL, multiprocessing e async/await	27
9.1	GIL na pratica	27
9.2	Multiprocessing	27
9.3	Custo do multiprocessing	28
9.4	ThreadPoolExecutor para GPX	28
9.5	Async/await	28
9.6	Onde async entra no projeto?	29
9.7	Ordem recomendada	29
10	Logging e profiling	30
10.1	Por que logging e essencial?	30
10.2	Configuracao de logger	30
10.3	Uso	30
10.4	Profiling simples com perf_counter	31
10.5	Throughput	31
10.6	Benchmark de batch size	31
10.7	Profiling mais avancado	31
11	Algoritmos e estruturas de dados aplicados	32
11.1	list / array	32
11.2	dict / hash map	32
11.3	set / hash set	32
11.4	queue	32
11.5	tree	33
11.6	recursion	33
11.7	sorting	33
11.8	binary search	33

11.9two pointers	33
11.10liding window	34
11.11O(n ²): o que evitar	34
12Testes e exercicios	35
12.1Por que testar com XML pequeno?	35
12.2Testes de funcoes puras	35
12.3Exercicios progressivos	36
12.3.1Exercicio 1: Counter	36
12.3.2Exercicio 2: batch	36
12.3.3Exercicio 3: queue	36
12.3.4Exercicio 4: threading	36
12.3.5Exercicio 5: profiling	36
12.3.6Exercicio 6: banco	36
12.4Testes de banco	37
13Roadmap passo a passo	38
13.1Milestone 1: Infra e Postgres	38
13.2Milestone 2: Profiler XML	38
13.3Milestone 3: Parser sincrono	39
13.4Milestone 4: Benchmark	39
13.5Milestone 5: Queue + Threading	39
13.6Milestone 6: GPX concorrente	40
13.7Milestone 7: Multiprocessing	40
13.8Milestone 8: Insights	40
13.9Milestone 9: Dashboard	41
14Codigo base sugerido para comecar	42
14.1src/app.py	42
14.2Comando para rodar	42
14.3Dependencias iniciais	42
14.4Queries iniciais de insight	43
14.4.1Tipos mais comuns	43
14.4.2Batimento medio por dia	43
14.4.3Passos por dia	43
14.4.4Media movel de 7 dias	43
14.5Comando de validacao com EXPLAIN	44
14.6Proxima tarefa pratica	44

Capítulo 1

Introducao: o que este projeto quer resolver

A ideia central do projeto e criar um aplicativo capaz de ler os dados exportados do Apple Health no iPhone e transformar esses arquivos em uma base estruturada para gerar insights pessoais. O ponto importante e que o arquivo exportado pode ser grande: centenas de megabytes de XML e varios arquivos GPX. Portanto, este nao e apenas um exercicio de leitura de arquivo. E um problema real de engenharia de dados local.

Objetivo principal

Criar uma pipeline que leia o Apple Health Export, processe os dados sem explodir memoria, salve em Postgres e permita consultas para gerar insights como batimento cardiaco medio, passos por dia, sono, treinos, calorias, distancia e tendencias ao longo do tempo.

O fluxo do projeto sera:

```
Apple Health export bruto
  -> streaming parser
  -> fila / batches
  -> Postgres
  -> queries otimizadas
  -> insights
  -> dashboard / app
```

A primeira decisao importante e: nao analisar o XML gigante diretamente toda vez. O XML e o dado bruto. Ele deve ser parseado uma vez para uma base estruturada. Depois disso, os insights devem consultar o banco.

1.1 Por que este projeto e bom para estudo?

Porque ele junta varios conceitos que normalmente aparecem separados:

- arquivos grandes e `streaming` ;
- escrita em banco e `I/O-bound` ;
- parsing e conversoes com parte `CPU-bound` ;
- `filas` , `threads` , `multiprocessing` e `async/await` ;

- `dataclass`, `batching`, `logging`, `profiling` ;
- estruturas de dados como `list`, `dict`, `set`, `queue` ;
- conceitos de Big O: `O(1)`, `O(n)`, `O(log n)`, `O(n²)` ;
- Postgres, indices, EXPLAIN ANALYZE e performance de queries.

Ou seja: o app vira um desafio pratico de engenharia em Python.

1.2 O que nao fazer

Evite carregar o XML inteiro em memoria:

```
1 content = file.read_text(encoding="utf-8")
2 soup = BeautifulSoup(content, "xml")
```

Essa abordagem pode funcionar para XML pequeno, mas e ruim para arquivos de 280 MB, 628 MB ou maiores. Ela carrega todo o texto e ainda monta uma arvore completa em memoria.

1.3 O que fazer

Usar parsing incremental:

```
1 import xml.etree.ElementTree as ET
2
3 for event, element in ET.iterparse(xml_file, events=("end",)):
4     # processa o elemento
5     element.clear()
```

Esse modelo le um pedaco, processa e limpa. A ideia e manter memoria quase constante.

1.4 Meta de aprendizado

A regra do projeto sera:

primeiro funciona → depois mede → depois otimiza → depois compara

Isso impede que voce comece com async, multiprocessing, threads, Postgres, indices e dashboard tudo ao mesmo tempo. Primeiro construiremos uma versao simples e correta. Depois adicionaremos complexidade de forma controlada.

Capítulo 2

Mapa dos conceitos que o projeto vai treinar

Este capítulo apresenta cada conceito rapidamente e mostra onde ele aparece no projeto. A ideia é você reconhecer o nome, entender a intuição e depois aplicar no código.

2.1 I/O-bound

I/O significa *Input/Output*. Um trecho é I/O-bound quando o programa passa boa parte do tempo esperando algo externo à CPU.

Exemplos:

- ler arquivo do disco;
- escrever arquivo;
- inserir dados no Postgres;
- consultar banco;
- fazer request HTTP;
- esperar uma resposta de rede.

No projeto:

ler export.xml	-> I/O
inserir no Postgres	-> I/O
ler arquivos .gpx	-> I/O
salvar logs	-> I/O

2.2 CPU-bound

Um trecho é CPU-bound quando o gargalo é computação. O programa está gastando tempo calculando, convertendo ou processando dados.

No projeto:

parsear XML em objetos Python	-> parcialmente CPU-bound
converter strings para datas	-> CPU-bound leve
calcular features e estatísticas	-> CPU-bound
normalizar milhões de pontos GPS	-> CPU-bound

2.3 GIL

GIL significa *Global Interpreter Lock*. No CPython, ele impede que varias threads executem bytecode Python puro ao mesmo tempo no mesmo processo.

Consequencia pratica:

- Threads ajudam bastante quando o gargalo e I/O.
- Threads geralmente nao aceleram muito trabalho CPU-bound puro.
- Multiprocessing ajuda CPU-bound porque cada processo tem seu proprio interpretador e seu proprio GIL.
- Async ajuda I/O concorrente, mas nao deixa calculo CPU-bound magicamente paralelo.

2.4 Stream

Stream e processar dados aos poucos. Em vez de carregar tudo, voce consome item por item.

```
1 for record in stream_records(xml_file):  
2     process(record)
```

No projeto, ET.iterparse sera a base do streaming.

2.5 Fila

Fila e uma estrutura FIFO: *first in, first out*. O primeiro item que entra e o primeiro que sai.

No projeto:

producer/parser -> Queue -> consumer/writer Postgres

A fila desacopla velocidades diferentes. Se o parser for mais rapido que o banco, a fila absorve temporariamente. Se a fila encher, o parser espera. Isso e **backpressure**.

2.6 Batching

Batching e agrupar varias operacoes em uma so.

Ruim:

```
1 registro -> INSERT  
1 registro -> INSERT  
1 registro -> INSERT
```

Melhor:

```
5000 registros -> INSERT batch  
5000 registros -> INSERT batch  
5000 registros -> INSERT batch
```

O algoritmo ainda e $O(n)$, mas o fator constante melhora muito.

2.7 Logging

Logging e registrar eventos importantes do programa. Neste projeto, logs devem mostrar:

- quantos registros foram processados;
- tamanho do batch;
- tamanho da fila;
- tempo por insert;
- registros por segundo;
- erros de parsing ou banco.

2.8 Profiling

Profiling e medir onde o programa gasta tempo. Sem profiling, otimizacao vira chute. Medidas importantes:

- tempo total;
- tempo por batch;
- registros por segundo;
- uso de memoria;
- query time no banco.

2.9 Big O aplicado

Complexidade	Aplicacao no projeto
$O(1)$	acesso medio em dict, set, append em lista
$O(n)$	percorrer todos os elementos do XML uma vez
$O(\log n)$	busca usando indice B-tree no Postgres
$O(n \log n)$	ordenacao de registros por data
$O(n^2)$	comparacoes ingênuas entre todos os pares de duas listas; algo a evitar

2.10 Resumo do mapa

Conceito	Onde aparece
list / array	batches de registros
dict / hash map	metadata, contadores, mapeamento de tipos
set / hash set	filtro rapido de tipos aceitos
queue	producer/consumer
tree	XML e uma arvore
stack	percursos DFS opcionais

recursion	exploracao de XML pequeno; evitar para XML gigante
sorting	ordenar por data
binary search	achar registros em intervalos ordenados
two pointers	cruzar intervalos de sono com batimentos
sliding window	medias moveis de 7/14 dias
caching	summary tables e materialized views
indexing	indices por tipo e data no Postgres

Capítulo 3

Estrutura do projeto

Voce ja tem uma estrutura inicial. A ideia e manter essa base e agregar os modulos necessarios aos poucos.

3.1 Estrutura inicial fornecida

```
apple_health_export
├── .editorconfig
├── README.md
├── notebooks
│   ├── README.md
│   └── playground.ipynb
├── poetry.lock
├── pyproject.toml
└── src
    ├── __init__.py
    ├── app.py
    ├── config
    │   ├── __init__.py
    │   ├── logger.py
    │   └── settings.py
    └── parser
        ├── __init__.py
        ├── base_parser.py
        └── xml_parser.py
```

Essa estrutura ja tem tres decisoes boas:

- src/app.py: ponto de entrada;
- src/config: configuracao e logging;
- src/parser: responsabilidade de parsing separada.

3.2 Estrutura agregada proposta

Agora vamos expandir sem destruir sua organizacao:

```
apple_health_export
├── .editorconfig
├── README.md
└── docker-compose.yml
```

```

├── .env.example
├── notebooks
│   ├── README.md
│   └── playground.ipynb
├── poetry.lock
├── pyproject.toml
├── tests
│   ├── __init__.py
│   ├── test_parser_utils.py
│   ├── test_xml_parser.py
│   └── fixtures
│       └── small_export.xml
├── src
│   ├── __init__.py
│   ├── app.py
│   ├── config
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── settings.py
│   ├── db
│   │   ├── __init__.py
│   │   ├── connection.py
│   │   ├── schema.py
│   │   └── repository.py
│   ├── models
│   │   ├── __init__.py
│   │   └── health_record.py
│   ├── parser
│   │   ├── __init__.py
│   │   ├── base_parser.py
│   │   ├── xml_parser.py
│   │   ├── gpx_parser.py
│   │   └── profiler.py
│   ├── pipelines
│   │   ├── __init__.py
│   │   ├── sync_pipeline.py
│   │   ├── threaded_pipeline.py
│   │   └── multiprocessing_pipeline.py
│   └── insights
│       ├── __init__.py
│       ├── daily_summary.py
│       └── queries.py

```

3.3 Por que cada pasta existe?

Pasta/arquivo	Responsabilidade
config/settings.py	centralizar variaveis como host do banco, porta, usuario, senha, batch size
config/logger.py	configurar logs padronizados
models/	dataclasses e objetos de dominio
parser/	transformar XML/GPX em objetos Python

db/	conexao, schema e escrita no Postgres
pipelines/	orquestrar parser, fila, batches e writer
insights/	queries e agregacoes para gerar metricas
tests/	testes pequenos sem usar o XML gigante

3.4 Regra de separacao

Cada modulo deve ter uma responsabilidade clara:

- Parser nao deve saber detalhes do Docker.
- Repository nao deve parsear XML.
- Logger nao deve conter regra de negocio.
- Pipeline orquestra componentes, mas nao deve concentrar toda logica.

Essa separacao facilita testes e torna a evolucao mais segura.

Capítulo 4

I/O e streaming: lendo arquivos grandes sem sofrer

4.1 O que é I/O?

I/O significa *Input/Output*. Em termos simples, é tudo que envolve entrada ou saída de dados fora da CPU. Ler um arquivo, escrever em disco, consultar banco e esperar uma resposta de rede são exemplos de I/O.

No seu projeto, as operações de I/O mais importantes são:

- ler `export.xml`;
- ler arquivos `.gpx`;
- inserir dados no Postgres;
- escrever logs;
- consultar tabelas de insights.

4.2 Por que o XML gigante muda a estratégia?

Se o arquivo tem 280 MB ou 628 MB, carregar tudo em memória pode ser ruim. Além do texto bruto, bibliotecas como BeautifulSoup criam objetos Python para representar a árvore XML inteira. Isso pode multiplicar o uso de memória.

Evite no arquivo gigante

```
1 content = file.read_text(encoding="utf-8")
2 soup = BeautifulSoup(content, "xml")
```

BeautifulSoup ainda pode ser útil para arquivos pequenos, testes e inspeções pontuais. Mas não deve ser o parser principal do export gigante.

4.3 Streaming com iterparse

A alternativa é usar `xml.etree.ElementTree.iterparse`. Ele percorre o XML elemento por elemento.

```
1 from pathlib import Path
2 from collections.abc import Iterator
3 import xml.etree.ElementTree as ET
```

```
4
5
6 def clean_tag(tag: str) -> str:
7     if "}" in tag:
8         return tag.split("}", 1)[1]
9     return tag
10
11
12 def stream_record_elements(xml_file: Path) -> Iterator[ET.Element]:
13     context = ET.iterparse(xml_file, events=("end",))
14
15     for _, element in context:
16         tag = clean_tag(element.tag)
17
18         if tag == "Record":
19             yield element
20
21         element.clear()
```

4.4 Explicando cada nomezinho

- Path: objeto da biblioteca pathlib para lidar com caminhos de arquivo de forma mais segura que string.
- Iterator: tipo que indica que a função retorna valores aos poucos.
- ET: apelido comum para xml.etree.ElementTree.
- iterparse: parser incremental de XML.
- events=("end",): processa o elemento quando ele fecha, ou seja, quando já temos seus atributos e filhos.
- yield: retorna um item sem encerrar a função; cria um generator.
- element.clear(): limpa o elemento da memória depois de processar.

4.5 Complexidade

Se o XML tem n elementos, o parser passa por cada elemento uma vez:

$$T(n) = O(n)$$

A memória tende a ser quase constante:

$$M(n) \approx O(1)$$

Isso não significa literalmente zero crescimento, mas significa que você não precisa manter o XML inteiro em memória.

4.6 Primeiro exercício

Crie um XML pequeno em `tests/fixtures/small_export.xml` e teste se sua stream encontra apenas tags Record.

```
1 def test_stream_record_elements_returns_records_only(tmp_path):
2     xml_file = tmp_path / "small_export.xml"
3     xml_file.write_text("""
4     <HealthData>
5         <Record type="HKQuantityTypeIdentifierHeartRate" value="90" />
6         <Workout workoutActivityType="HKWorkoutActivityTypeRunning" />
7     </HealthData>
8     """)
9
10    records = list(stream_record_elements(xml_file))
11
12    assert len(records) == 1
13    assert records[0].attrib["type"] == "HKQuantityTypeIdentifierHeartRate"
```


Capítulo 5

Dataclass e modelagem de dados

5.1 Por que modelar antes de inserir no banco?

Ao parsear XML, voce recebe atributos como strings soltas. Se voce passar dicionarios por todo o projeto, o codigo fica fragil:

```
1 record["startDate"]
2 record["value"]
3 record["sourceName"]
```

O problema e que erros de chave aparecem tarde. Alem disso, nao fica claro quais campos existem.

Com dataclass, voce cria uma estrutura explicita:

```
1 record.start_date
2 record.value_numeric
3 record.source_name
```

5.2 Modelo HealthRecord

Arquivo sugerido: src/models/health_record.py

```
1 from dataclasses import dataclass
2
3
4 @dataclass(slots=True)
5 class HealthRecord:
6     apple_record_id: int
7     type: str | None
8     source_name: str | None
9     source_version: str | None
10    device: str | None
11    unit: str | None
12    creation_date: str | None
13    start_date: str | None
14    end_date: str | None
15    value: str | None
16    value_numeric: float | None
17    metadata: dict[str, str]
```

5.3 Explicando cada nomezinho

- @dataclass: decorator que gera automaticamente metodos como `__init__` e `__repr__`.

- `slots=True`: reduz overhead de memoria removendo o `__dict__` dinamico de cada instancia.
- `apple_record_id`: id sequencial criado pelo seu parser, diferente do id do banco.
- `type`: tipo do Record do Apple Health, por exemplo `HKQuantityTypeIdentifierHeartRate`.
- `source_name`: origem do dado, por exemplo Apple Watch ou iPhone.
- `value`: valor original em texto.
- `value_numeric`: tentativa de conversao para numero.
- `metadata`: informacoes extras de `MetadataEntry`.

5.4 Por que manter `value` e `value_numeric`?

Nem todo valor do Apple Health e numerico. Alguns sao categorias.

Numericos:

92
1200
75.4

Categoricos:

`HKCategoryValueSleepAnalysisAsleepCore`
`HKCategoryValueSleepAnalysisInBed`

Por isso salvamos os dois:

- `value`: preserva o valor original;
- `value_numeric`: permite agregacoes numericas quando possivel.

5.5 Funcao `safe_float`

```
1 def safe_float(value: str | None) -> float | None:
2     if value is None:
3         return None
4
5     try:
6         return float(value)
7     except ValueError:
8         return None
```

Ela tenta converter texto para numero. Se nao conseguir, retorna `None`. Isso evita quebrar o parser quando encontrar valores categoricos.

5.6 Exercicio

Implemente testes:

```
1 def test_safe_float_with_number():
2     assert safe_float("92.5") == 92.5
3
4
5 def test_safe_float_with_none():
6     assert safe_float(None) is None
7
8
9 def test_safe_float_with_text():
10    assert safe_float("HKCategoryValueSleepAnalysisAsleepCore") is None
```

Capítulo 6

Postgres desde o inicio

6.1 Por que Postgres em vez de CSV?

CSV é simples para debugar, mas o objetivo deste projeto também é treinar banco, índices, queries e performance. Então faz sentido usar Postgres desde o início.

Com Postgres você treina:

- schema design;
- tipos de dados;
- JSONB;
- batch insert;
- transações;
- índices;
- EXPLAIN ANALYZE;
- queries de insight.

6.2 Docker Compose para Postgres

Arquivo: docker-compose.yml

```
1 services:
2   postgres:
3     image: postgres:16
4     container_name: apple_health_postgres
5     environment:
6       POSTGRES_USER: apple
7       POSTGRES_PASSWORD: apple
8       POSTGRES_DB: apple_health
9     ports:
10      - "5433:5432"
11     volumes:
12      - postgres_data:/var/lib/postgresql/data
13
14 volumes:
15   postgres_data:
```

Subir banco:

```
1 docker compose up -d
```

6.3 Settings

Arquivo: src/config/settings.py

```

1 from dataclasses import dataclass
2 import os
3
4
5 @dataclass(frozen=True, slots=True)
6 class Settings:
7     postgres_host: str = os.getenv("POSTGRES_HOST", "localhost")
8     postgres_port: int = int(os.getenv("POSTGRES_PORT", "5433"))
9     postgres_user: str = os.getenv("POSTGRES_USER", "apple")
10    postgres_password: str = os.getenv("POSTGRES_PASSWORD", "apple")
11    postgres_db: str = os.getenv("POSTGRES_DB", "apple_health")
12    batch_size: int = int(os.getenv("BATCH_SIZE", "5000"))
13
14    @property
15    def database_url(self) -> str:
16        return (
17            f"postgresql://{self.postgres_user}:"
18            f"{self.postgres_password}@"
19            f"{self.postgres_host}:{self.postgres_port}/"
20            f"{self.postgres_db}"
21        )

```

6.4 Schema inicial

Tabela principal: health_records.

```

1 CREATE TABLE IF NOT EXISTS health_records (
2     id BIGSERIAL PRIMARY KEY,
3     apple_record_id BIGINT NOT NULL,
4     type TEXT,
5     source_name TEXT,
6     source_version TEXT,
7     device TEXT,
8     unit TEXT,
9     creation_date TIMESTAMPTZ,
10    start_date TIMESTAMPTZ,
11    end_date TIMESTAMPTZ,
12    value TEXT,
13    value_numeric DOUBLE PRECISION,
14    metadata JSONB
15 );

```

6.5 Tabela parse_runs

Essa tabela guarda historico de execucoes do parser.

```

1 CREATE TABLE IF NOT EXISTS parse_runs (
2     id BIGSERIAL PRIMARY KEY,
3     started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
4     finished_at TIMESTAMPTZ,
5     file_path TEXT NOT NULL,
6     total_records BIGINT DEFAULT 0,
7     status TEXT NOT NULL DEFAULT 'running',
8     error TEXT

```

```
9 );
```

6.6 Indices

Consultas de insights vao filtrar muito por tipo e data. Entao os primeiros indices sao:

```
1 CREATE INDEX IF NOT EXISTS idx_health_records_type
2 ON health_records (type);
3
4 CREATE INDEX IF NOT EXISTS idx_health_records_start_date
5 ON health_records (start_date);
6
7 CREATE INDEX IF NOT EXISTS idx_health_records_type_start_date
8 ON health_records (type, start_date);
```

6.7 Por que indice importa?

Sem indice, o banco pode precisar varrer muitas linhas:

$$O(n)$$

Com indice B-tree, buscas por tipo/data podem se aproximar de:

$$O(\log n)$$

Na pratica, confirme com:

```
1 EXPLAIN ANALYZE
2 SELECT
3     DATE(start_date) AS day,
4     AVG(value_numeric) AS avg_heart_rate
5 FROM health_records
6 WHERE type = 'HKQuantityTypeIdentifierHeartRate'
7 GROUP BY DATE(start_date)
8 ORDER BY day;
```

Capítulo 7

Parser streaming + batching

7.1 Fluxo desta etapa

Agora juntamos streaming, dataclass e Postgres.

XML element

- > parse attributes
- > HealthRecord dataclass
- > batch list
- > Postgres

7.2 Parser de elemento Record

Arquivo: src/parser/xml_parser.py

```
1 import xml.etree.ElementTree as ET
2 from src.models.health_record import HealthRecord
3
4
5 def clean_tag(tag: str) -> str:
6     if "}" in tag:
7         return tag.split("}", 1)[1]
8     return tag
9
10
11 def safe_float(value: str | None) -> float | None:
12     if value is None:
13         return None
14
15     try:
16         return float(value)
17     except ValueError:
18         return None
19
20
21 def extract_metadata(element: ET.Element) -> dict[str, str]:
22     metadata: dict[str, str] = {}
23
24     for child in element:
25         if clean_tag(child.tag) != "MetadataEntry":
26             continue
27
28         key = child.attrib.get("key")
29         value = child.attrib.get("value")
30
```

```

31     if key is not None and value is not None:
32         metadata[key] = value
33
34     return metadata
35
36
37 def parse_record_element(
38     element: ET.Element,
39     record_id: int,
40 ) -> HealthRecord:
41     raw_value = element.attrib.get("value")
42
43     return HealthRecord(
44         apple_record_id=record_id,
45         type=element.attrib.get("type"),
46         source_name=element.attrib.get("sourceName"),
47         source_version=element.attrib.get("sourceVersion"),
48         device=element.attrib.get("device"),
49         unit=element.attrib.get("unit"),
50         creation_date=element.attrib.get("creationDate"),
51         start_date=element.attrib.get("startDate"),
52         end_date=element.attrib.get("endDate"),
53         value=raw_value,
54         value_numeric=safe_float(raw_value),
55         metadata=extract_metadata(element),
56     )

```

7.3 Repository com batch insert

Instale o driver:

```
1 poetry add "psycopg[binary]"
```

Arquivo: src/db/repository.py

```

1 import json
2 from collections.abc import Sequence
3
4 import psycopg
5
6 from src.models.health_record import HealthRecord
7
8
9 class HealthRecordRepository:
10     def __init__(self, connection: psycopg.Connection) -> None:
11         self.connection = connection
12
13     def insert_batch(self, records: Sequence[HealthRecord]) -> None:
14         if not records:
15             return
16
17         rows = [
18             (
19                 record.apple_record_id,
20                 record.type,
21                 record.source_name,
22                 record.source_version,
23                 record.device,
24                 record.unit,
25                 record.creation_date,

```



```

26         record.start_date,
27         record.end_date,
28         record.value,
29         record.value_numeric,
30         json.dumps(record.metadata),
31     )
32     for record in records
33 ]
34
35 with self.connection.cursor() as cursor:
36     cursor.executemany(
37         """
38         INSERT INTO health_records (
39             apple_record_id,
40             type,
41             source_name,
42             source_version,
43             device,
44             unit,
45             creation_date,
46             start_date,
47             end_date,
48             value,
49             value_numeric,
50             metadata
51         )
52         VALUES (
53             %S, %S, %S, %S, %S, %S,
54             %S, %S, %S, %S, %S, %S
55         )
56         """,
57         rows,
58     )
59
60     self.connection.commit()

```

7.4 Pipeline sincrona

Arquivo: src/pipelines/sync_pipeline.py

```

1  from pathlib import Path
2  from time import perf_counter
3  import logging
4  import xml.etree.ElementTree as ET
5
6  from src.db.repository import HealthRecordRepository
7  from src.parser.xml_parser import clean_tag, parse_record_element
8
9  logger = logging.getLogger(__name__)
10
11
12 def run_sync_pipeline(
13     xml_file: Path,
14     repository: HealthRecordRepository,
15     batch_size: int,
16 ) -> None:
17     batch = []
18     record_id = 0
19     started_at = perf_counter()

```

```
20 context = ET.iterparse(xml_file, events=("end",))
21
22 for _, element in context:
23     tag = clean_tag(element.tag)
24
25     if tag == "Record":
26         record_id += 1
27         batch.append(parse_record_element(element, record_id))
28
29     if len(batch) >= batch_size:
30         insert_started_at = perf_counter()
31         repository.insert_batch(batch)
32         insert_elapsed = perf_counter() - insert_started_at
33
34         logger.info(
35             "Inserted %s records in %.2fs | total=%s",
36             len(batch),
37             insert_elapsed,
38             record_id,
39         )
40
41         batch.clear()
42
43     element.clear()
44
45 if batch:
46     repository.insert_batch(batch)
47
48 elapsed = perf_counter() - started_at
49 logger.info(
50     "Finished parsing %s records in %.2fs | %.2f records/s",
51     record_id,
52     elapsed,
53     record_id / elapsed if elapsed > 0 else 0,
54 )
55 )
```

7.5 Conceitos aplicados

- batch: uma list; append é O(1) médio.
- batch_size: tradeoff entre memória e velocidade.
- iterparse: lazy loading / streaming.
- insert_batch: reduz overhead de I/O com o banco.
- perf_counter: mede tempo com precisão suficiente para benchmark.

7.6 Experimento

Rode a pipeline com:

```
batch_size = 500
batch_size = 5000
batch_size = 20000
batch_size = 50000
```

Anote:

- tempo total;
- records/s;
- memoria percebida;
- tempo medio por batch.

Capítulo 8

Filas e threads: producer/consumer

8.1 Por que usar fila?

Parser e banco podem ter velocidades diferentes. O parser pode produzir registros mais rapido do que o Postgres consegue inserir, ou o banco pode ficar esperando I/O enquanto o parser poderia continuar trabalhando.

A arquitetura producer/consumer separa essas responsabilidades:

```
producer thread
    le XML e cria HealthRecord
    -> Queue
consumer thread
    pega HealthRecord e insere em batch no Postgres
```

8.2 Queue

queue.Queue e uma fila thread-safe da biblioteca padrao.

```
1 from queue import Queue
2
3 record_queue: Queue[HealthRecord | None] = Queue(maxsize=10_000)
```

8.3 Explicando cada nomezinho

- Queue: fila segura para multiplas threads.
- maxsize: limite da fila. Evita crescimento infinito de memoria.
- HealthRecord | None: a fila aceita registros ou None.
- None: usado como sentinela para sinalizar fim do trabalho.
- backpressure: quando a fila enche, o producer bloqueia. Isso protege a memoria.

8.4 Producer

```
1 from pathlib import Path
2 from queue import Queue
3 import xml.etree.ElementTree as ET
4
5 from src.models.health_record import HealthRecord
6 from src.parser.xml_parser import clean_tag, parse_record_element
```

```
7
8
9 def producer(
10     xml_file: Path,
11     record_queue: Queue[HealthRecord | None],
12 ) -> None:
13     record_id = 0
14     context = ET.iterparse(xml_file, events=("end",))
15
16     for _, element in context:
17         if clean_tag(element.tag) == "Record":
18             record_id += 1
19             record = parse_record_element(element, record_id)
20             record_queue.put(record)
21
22     element.clear()
23
24     record_queue.put(None)
```

8.5 Consumer

```
1 from queue import Queue
2
3 from src.db.repository import HealthRecordRepository
4 from src.models.health_record import HealthRecord
5
6
7 def consumer(
8     record_queue: Queue[HealthRecord | None],
9     repository: HealthRecordRepository,
10     batch_size: int,
11 ) -> None:
12     batch: list[HealthRecord] = []
13
14     while True:
15         record = record_queue.get()
16
17         if record is None:
18             break
19
20         batch.append(record)
21
22         if len(batch) >= batch_size:
23             repository.insert_batch(batch)
24             batch.clear()
25
26     if batch:
27         repository.insert_batch(batch)
```

8.6 Rodando com Thread

```
1 from pathlib import Path
2 from queue import Queue
3 from threading import Thread
4
5 from src.db.repository import HealthRecordRepository
```

```
6 from src.models.health_record import HealthRecord
7
8
9 def run_threaded_pipeline(
10     xml_file: Path,
11     repository: HealthRecordRepository,
12     batch_size: int,
13 ) -> None:
14     record_queue: Queue[HealthRecord | None] = Queue(maxsize=10_000)
15
16     producer_thread = Thread(
17         target=producer,
18         args=(xml_file, record_queue),
19         name="xml-producer",
20     )
21
22     consumer_thread = Thread(
23         target=consumer,
24         args=(record_queue, repository, batch_size),
25         name="postgres-consumer",
26     )
27
28     producer_thread.start()
29     consumer_thread.start()
30
31     producer_thread.join()
32     consumer_thread.join()
```

8.7 Quando thread ajuda?

Threads ajudam quando existe espera de I/O. No projeto, escrever no Postgres e ler arquivos são I/O-bound. Enquanto uma thread espera o banco responder, outra pode continuar lendo ou preparando dados.

Mas se o gargalo principal for parse CPU-bound, threads podem não acelerar muito por causa do GIL.

8.8 Experimento

Compare:

- pipeline síncrona;
- pipeline com producer/consumer e threads;
- batch sizes diferentes;
- queue maxsize diferente.

Pergunta que você deve responder: **thread melhorou por causa de I/O ou não mudou por causa de CPU/GIL?**

Capítulo 9

GIL, multiprocessing e async/await

9.1 GIL na pratica

O GIL impede que multiplas threads executem bytecode Python puro simultaneamente no mesmo processo. Isso nao significa que threads sao inuteis. Significa que voce precisa entender o tipo de gargalo.

Cenario	Ferramenta provavel
I/O-bound	threads ou async
CPU-bound	multiprocessing
Muitos arquivos pequenos	ThreadPoolExecutor pode ajudar
Transformacao numerica pesada	ProcessPoolExecutor pode ajudar
API web com muitas esperas	async/await pode ajudar

9.2 Multiprocessing

Multiprocessing cria processos separados. Cada processo tem seu proprio interpretador Python e seu proprio GIL. Por isso ele pode acelerar tarefas CPU-bound.

Exemplo didatico:

```
1 from concurrent.futures import ProcessPoolExecutor
2
3
4 def cpu_heavy_transform(value: float | None) -> float | None:
5     if value is None:
6         return None
7
8     result = value
9     for _ in range(100_000):
10         result = (result * 1.000001) / 1.000001
11
12     return result
13
14
15 def process_values(values: list[float | None]) -> list[float | None]:
16     with ProcessPoolExecutor() as executor:
17         return list(executor.map(cpu_heavy_transform, values))
```

9.3 Custo do multiprocessing

Multiprocessing nao e gratis:

- cada processo consome mais memoria;
- dados precisam ser serializados entre processos;
- criar processos tem overhead;
- para tarefas pequenas, pode ficar mais lento.

Use multiprocessing quando o trabalho por item for pesado o suficiente.

9.4 ThreadPoolExecutor para GPX

Arquivos GPX sao bons candidatos para concorrência porque geralmente existem varios arquivos menores.

```
1 from concurrent.futures import ThreadPoolExecutor
2 from pathlib import Path
3
4
5 def parse_gpx_file(file: Path) -> list[dict[str, str | None]]:
6     # parseia um arquivo GPX e retorna pontos
7     return []
8
9
10 def parse_many_gpx(files: list[Path]) -> list[dict[str, str | None]]:
11     rows: list[dict[str, str | None]] = []
12
13     with ThreadPoolExecutor(max_workers=8) as executor:
14         for file_rows in executor.map(parse_gpx_file, files):
15             rows.extend(file_rows)
16
17     return rows
```

9.5 Async/await

async/await e uma forma de concorrência cooperativa. Em vez de criar varias threads, um event loop alterna entre tarefas quando elas fazem await em operacoes I/O.

Exemplo minimo:

```
1 import asyncio
2
3
4 async def producer(queue: asyncio.Queue[int]) -> None:
5     for item in range(10):
6         await queue.put(item)
7
8     await queue.put(-1)
9
10
11 async def consumer(queue: asyncio.Queue[int]) -> None:
12     while True:
13         item = await queue.get()
14
15         if item == -1:
```



```
16         break
17
18     print(item)
19
20
21 async def main() -> None:
22     queue: asyncio.Queue[int] = asyncio.Queue()
23
24     await asyncio.gather(
25         producer(queue),
26         consumer(queue),
27     )
28
29
30 asyncio.run(main())
```

9.6 Onde async entra no projeto?

Nao comece com async no parser principal. Primeiro faca a versao sincrona eficiente. Async pode entrar depois em:

- API com FastAPI;
- consulta de status de parse;
- jobs em background;
- writer async com asyncpg;
- dashboard consultando dados sem bloquear.

9.7 Ordem recomendada

1. parser sincrono;
2. logs e profiling;
3. threads com queue;
4. ThreadPool para GPX;
5. ProcessPool para CPU-bound;
6. async para API ou I/O concorrente futuro.

Capítulo 10

Logging e profiling

10.1 Por que logging e essencial?

Sem logs, voce nao sabe se o parser esta travado, lento, quebrando em algum Record especifico ou inserindo pouco por segundo.

Logs devem responder:

- quantos registros ja foram processados?
- quantos registros por segundo?
- qual o tamanho do batch?
- quanto tempo cada insert demorou?
- qual thread esta executando?
- ocorreu erro em qual arquivo?

10.2 Configuracao de logger

Arquivo: src/config/logger.py

```
1 import logging
2 import sys
3
4
5 def configure_logging(level: int = logging.INFO) -> None:
6     logging.basicConfig(
7         level=level,
8         format=(
9             "%(asctime)s | %(levelname)s | "
10            "%(threadName)s | %(name)s | %(message)s"
11        ),
12         handlers=[logging.StreamHandler(sys.stdout)],
13     )
```

10.3 Uso

```
1 import logging
2
3 logger = logging.getLogger(__name__)
4
```

```
5 logger.info("Starting Apple Health parser")
6 logger.warning("Skipping unsupported tag: %s", tag)
7 logger.exception("Failed to insert batch")
```

10.4 Profiling simples com perf_counter

```
1 from time import perf_counter
2
3 started_at = perf_counter()
4
5 # operacao que voce quer medir
6
7 elapsed = perf_counter() - started_at
8 print(f"Elapsed: {elapsed:.2f}s")
```

10.5 Throughput

Throughput é taxa de processamento. Neste projeto, uma métrica boa é registros por segundo.

```
1 records_per_second = total_records / elapsed
```

Exemplo de log desejado:

2026-06-23 08:00:00 | INFO | MainThread | parser | Inserted 5000 records in 0.41s

10.6 Benchmark de batch size

Crie uma tabela manual:

Batch size	Tempo total	Records/s	Observacao
500			muitos inserts
5000			equilibrio inicial
20000			menos commits
50000			maior memoria

10.7 Profiling mais avancado

Depois voce pode usar:

```
1 python -m cProfile -o profile.out src/app.py parse
```

E analisar com:

```
1 python -m pstats profile.out
```

Mas comece com `perf_counter`, porque ele te obriga a formular exatamente o que está medindo.

Capítulo 11

Algoritmos e estruturas de dados aplicados

11.1 list / array

Em Python, list é um array dinâmico. No projeto, ela aparece como batch:

```
1 batch: list[HealthRecord] = []  
2 batch.append(record)
```

append é $O(1)$ médio. Acesso por índice também é $O(1)$.

11.2 dict / hash map

dict é uma tabela hash. Usamos para metadata e contadores.

```
1 metadata: dict[str, str] = {}  
2 metadata[key] = value
```

Lookup médio:

$O(1)$

11.3 set / hash set

set é usado para membership rápido.

```
1 allowed_types = {  
2     "HKQuantityTypeIdentifierHeartRate",  
3     "HKQuantityTypeIdentifierStepCount",  
4 }  
5  
6 if record.type in allowed_types:  
7     process(record)
```

Checar se um item está no set é $O(1)$ médio.

11.4 queue

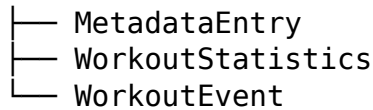
Fila FIFO usada em producer/consumer.

```
1 record_queue.put(record)  
2 record = record_queue.get()
```

11.5 tree

XML é uma árvore:

Workout



Cada tag é um nó; tags internas são filhos.

11.6 recursion

Recursão pode percorrer árvores pequenas:

```
1 def walk(element):
2     print(element.tag)
3
4     for child in element:
5         walk(child)
```

Mas evite usar recursão carregando a árvore inteira do XML gigante. Para o export real, prefira streaming.

11.7 sorting

Ordenar por data é comum em séries temporais. Ordenação geralmente custa:

$$O(n \log n)$$

No banco:

```
1 SELECT *
2 FROM health_records
3 ORDER BY start_date;
```

11.8 binary search

Busca binária encontra um item em lista ordenada em $O(\log n)$. Em Python:

```
1 from bisect import bisect_left
2
3 positions = [1, 3, 5, 7, 9]
4 index = bisect_left(positions, 6)
```

No projeto, a ideia aparece ao buscar registros em séries temporais ordenadas. No banco, índices B-tree usam uma lógica parecida de busca eficiente.

11.9 two pointers

Dois ponteiros servem para cruzar duas listas ordenadas sem fazer $O(n^2)$.

Exemplo: achar batimentos dentro de intervalos de sono.

sono: [inicio, fim]
heart rate: timestamp ordenado

Em vez de comparar todo sono com todo batimento, você avança ponteiros conforme as datas.

11.10 sliding window

Sliding window é uma janela movel. Exemplo: media movel de 7 dias.

Em SQL:

```
1 SELECT
2     day,
3     steps,
4     AVG(steps) OVER (
5         ORDER BY day
6         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
7     ) AS steps_7d_avg
8 FROM daily_summary
9 ORDER BY day;
```

Isso é muito útil para tendencias de passos, sono e batimento.

11.11 $O(n^2)$: o que evitar

Evite loops aninhados sem necessidade:

```
1 for sleep_interval in sleep_intervals:
2     for heart_rate in heart_rates:
3         compare(sleep_interval, heart_rate)
```

Se cada lista tem muitos itens, isso vira $O(n^2)$. Busque alternativas com ordenacao, indices, SQL, two pointers ou janelas.

Capítulo 12

Testes e exercicios

12.1 Por que testar com XML pequeno?

Voce nao deve testar com o arquivo gigante. Testes devem ser pequenos, rapidos e previsiveis.

Crie:

tests/fixtures/small_export.xml

Conteudo exemplo:

```
1 <HealthData>
2   <ExportDate value="2026-06-23 08:00:00 -0300" />
3   <Record
4     type="HKQuantityTypeIdentifierHeartRate"
5     sourceName="Apple Watch"
6     unit="count/min"
7     startDate="2026-06-23 07:00:00 -0300"
8     endDate="2026-06-23 07:00:00 -0300"
9     value="90"
10  />
11  <Record
12    type="HKCategoryTypeIdentifierSleepAnalysis"
13    sourceName="Apple Watch"
14    startDate="2026-06-23 00:00:00 -0300"
15    endDate="2026-06-23 07:00:00 -0300"
16    value="HKCategoryValueSleepAnalysisAsleepCore"
17  />
18 </HealthData>
```

12.2 Testes de funcoes puras

```
1 from src.parser.xml_parser import clean_tag, safe_float
2
3
4 def test_clean_tag_without_namespace():
5     assert clean_tag("Record") == "Record"
6
7
8 def test_clean_tag_with_namespace():
9     tag = "{http://www.topografix.com/GPX/1/1}trkpt"
10    assert clean_tag(tag) == "trkpt"
11
12
```

```
13 def test_safe_float_number():
14     assert safe_float("90") == 90.0
15
16
17 def test_safe_float_text():
18     assert safe_float("HKCategoryValueSleepAnalysisAsleepCore") is None
```

12.3 Exercícios progressivos

12.3.1 Exercício 1: Counter

Conte palavras:

```
1 words = ["heart", "steps", "heart", "sleep"]
```

Resultado esperado:

```
1 {"heart": 2, "steps": 1, "sleep": 1}
```

Depois aplique a mesma ideia para contar tipos de Record.

12.3.2 Exercício 2: batch

Crie uma função que recebe números de 1 a 20 e separa em batches de 5.

Entrada:

```
1 list(range(1, 21))
```

Saída:

```
1 [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], ...]
```

12.3.3 Exercício 3: queue

Crie um producer que coloca números em uma fila e um consumer que imprime os números.

12.3.4 Exercício 4: threading

Rode producer e consumer em threads separadas e coloque o nome da thread no log.

12.3.5 Exercício 5: profiling

Meça o tempo de uma função usando perf_counter.

12.3.6 Exercício 6: banco

Insira 1000 registros fake no Postgres com:

- insert linha por linha;
- batch insert;

Compare o tempo.

12.4 Testes de banco

No começo, pode testar manualmente. Depois, use um banco de teste via Docker.

Teste importante:

- inserir batch;
- contar linhas;
- consultar por tipo;
- garantir que `value_numeric` vira NULL quando valor e texto.

Capítulo 13

Roadmap passo a passo

13.1 Milestone 1: Infra e Postgres

Entregaveis:

- `docker-compose.yml`;
- `.env.example`;
- `settings.py`;
- conexao com Postgres;
- schema inicial.

Conceitos:

- banco relacional;
- connection string;
- variaveis de ambiente;
- tipos SQL;
- indices.

13.2 Milestone 2: Profiler XML

Objetivo: descobrir o que existe no export.

Entregaveis:

- contar tags;
- contar tipos de Record;
- coletar amostras pequenas;
- salvar resultado em JSON ou tabela.

Conceitos:

- streaming;
- Counter;
- dict;
- set;
- $O(n)$.

13.3 Milestone 3: Parser síncrono

Entregáveis:

- HealthRecord dataclass;
- parser com `iterparse`;
- batch insert;
- logs de progresso;
- testes com XML pequeno.

Conceitos:

- dataclass;
- list como batch;
- memory vs speed;
- I/O-bound;
- logging.

13.4 Milestone 4: Benchmark

Entregáveis:

- medir tempo total;
- medir records/s;
- comparar batch sizes;
- salvar resultados em tabela ou markdown.

Conceitos:

- profiling;
- throughput;
- fator constante;
- tradeoffs.

13.5 Milestone 5: Queue + Threading

Entregáveis:

- producer;
- consumer;
- `queue.Queue`;
- backpressure com `maxsize`;

- comparar com pipeline sincrona.

Conceitos:

- threads;
- GIL;
- I/O-bound;
- concorrencia;
- fila FIFO.

13.6 Milestone 6: GPX concorrente

Entregaveis:

- parser de GPX;
- ThreadPoolExecutor;
- insert batch de pontos;
- benchmark por numero de workers.

Conceitos:

- fan-out/fan-in;
- I/O concorrente;
- threads para muitos arquivos.

13.7 Milestone 7: Multiprocessing

Entregaveis:

- transformacao CPU-bound artificial;
- benchmark sync vs threads vs processos;
- conclusao sobre GIL.

13.8 Milestone 8: Insights

Entregaveis:

- batimento medio por dia;
- passos por dia;
- treinos por semana;
- sono por noite;
- media movel de 7 dias;
- tabela daily_health_summary.

13.9 Milestone 9: Dashboard

Depois que os dados estiverem no banco e os insights basicos existirem, crie uma interface com Streamlit ou FastAPI + frontend.

Ordem recomendada:

1. Streamlit para visualizar rapido;
2. FastAPI se quiser API real;
3. async depois que houver endpoints I/O-bound claros.

Capítulo 14

Codigo base sugerido para come- car

Este capítulo junta um esqueleto minimo para iniciar sem perder sua estrutura.

14.1 src/app.py

```
1 from pathlib import Path
2 import psycpg
3
4 from src.config.logger import configure_logging
5 from src.config.settings import Settings
6 from src.db.repository import HealthRecordRepository
7 from src.pipelines.sync_pipeline import run_sync_pipeline
8
9
10 def main() -> None:
11     configure_logging()
12     settings = Settings()
13
14     xml_file = Path("data/export.xml").resolve()
15
16     with psycpg.connect(settings.database_url) as connection:
17         repository = HealthRecordRepository(connection)
18
19         run_sync_pipeline(
20             xml_file=xml_file,
21             repository=repository,
22             batch_size=settings.batch_size,
23         )
24
25
26 if __name__ == "__main__":
27     main()
```

14.2 Comando para rodar

```
1 poetry run python src/app.py
```

14.3 Dependencias iniciais

```
1 poetry add "psycpg[binary]"
2 poetry add --group dev pytest
```

Mais tarde:

```
1 poetry add pandas streamlit plotly
```

14.4 Queries iniciais de insight

14.4.1 Tipos mais comuns

```
1 SELECT
2     type,
3     COUNT(*) AS total
4 FROM health_records
5 GROUP BY type
6 ORDER BY total DESC;
```

14.4.2 Batimento medio por dia

```
1 SELECT
2     DATE(start_date) AS day,
3     AVG(value_numeric) AS avg_heart_rate,
4     MIN(value_numeric) AS min_heart_rate,
5     MAX(value_numeric) AS max_heart_rate,
6     COUNT(*) AS samples
7 FROM health_records
8 WHERE type = 'HKQuantityTypeIdentifierHeartRate'
9 GROUP BY DATE(start_date)
10 ORDER BY day;
```

14.4.3 Passos por dia

```
1 SELECT
2     DATE(start_date) AS day,
3     SUM(value_numeric) AS steps
4 FROM health_records
5 WHERE type = 'HKQuantityTypeIdentifierStepCount'
6 GROUP BY DATE(start_date)
7 ORDER BY day;
```

14.4.4 Media movel de 7 dias

```
1 WITH daily_steps AS (
2     SELECT
3         DATE(start_date) AS day,
4         SUM(value_numeric) AS steps
5     FROM health_records
6     WHERE type = 'HKQuantityTypeIdentifierStepCount'
7     GROUP BY DATE(start_date)
8 )
9 SELECT
10     day,
11     steps,
```

```
12     AVG(steps) OVER (  
13         ORDER BY day  
14         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
15     ) AS steps_7d_avg  
16 FROM daily_steps  
17 ORDER BY day;
```

14.5 Comando de validacao com EXPLAIN

```
1 EXPLAIN ANALYZE  
2 SELECT  
3     DATE(start_date) AS day,  
4     AVG(value_numeric) AS avg_heart_rate  
5 FROM health_records  
6 WHERE type = 'HKQuantityTypeIdentifierHeartRate'  
7 GROUP BY DATE(start_date)  
8 ORDER BY day;
```

Rode antes e depois dos indices para observar o plano de execucao.

14.6 Proxima tarefa pratica

A proxima tarefa deve ser implementar a Milestone 1:

1. criar docker-compose.yml;
2. criar tabelas;
3. criar Settings;
4. criar configure_logging;
5. criar HealthRecord;
6. criar testes de clean_tag e safe_float;
7. rodar um parse pequeno com fixture.

So depois disso processe o arquivo gigante.

Conclusao

Este documento transforma seu parser do Apple Health em um laboratorio pratico de engenharia: primeiro funciona, depois mede, depois otimiza e compara. A implementacao final deve permitir gerar insights pessoais a partir de dados reais, enquanto voce treina streaming, filas, Postgres, logging, profiling, concorrencia e estruturas de dados em Python.